



TECHNICAL PROJECT PLAN

Website Redesign & AI Assistant Implementation

A Six-Month Technical Delivery Plan

This document defines the end-to-end plan to design, build, test and launch a new CUS Solution web platform (modeled on cussolution.com) with an integrated, AI-powered virtual career assistant. It covers UX/UI design, frontend and backend engineering, quality assurance, the conversational AI (RAG) architecture, and a dedicated chatbot evaluation program — sequenced across six calendar months.

Project	CUS Solution — Web & AI Assistant Platform
Reference site	https://cussolution.com/
Duration	6 months (Month 1 – Month 6)
Audience	Engineering, QA, ML and Delivery teams
Prepared for	Snehith Reyya, CUS Solution
Version / Status	v1.0 — For technical review

1. Executive Summary

CUS Solution provides IT career coaching and staffing services. The current public site (cussolution.com) presents services, testimonials and a blog, but relies on static content and manual contact forms for lead capture and candidate support. This project delivers a rebuilt, performance-focused website with the same information architecture and an added **AI virtual assistant** that answers candidate and client questions, qualifies leads, recommends services, and books consultations — 24/7.

The engagement is structured into six one-month phases. Each phase has explicit entry criteria, engineering deliverables, and exit gates. Design, coding, and testing run as overlapping tracks rather than a strict waterfall; the AI assistant is developed as a parallel workstream from Month 3 and hardened through a dedicated evaluation phase in Month 5 before the combined launch in Month 6.

Primary objectives

- Rebuild the marketing site with a modern, responsive, accessible (WCAG 2.1 AA) front end that mirrors the existing site map: Home, About, Services (Career Change, Staffing, Outsourced HR, Hiring Managers), Our Approach, Blog, Contact.
- Introduce an AI career assistant (chat widget) grounded in CUS content and services via Retrieval-Augmented Generation (RAG), with lead-capture and appointment-booking actions.
- Establish an automated testing and CI/CD foundation covering unit, integration, end-to-end, accessibility, performance, security, and a separate conversational-quality suite for the chatbot.
- Ship a measurable, analytics-instrumented platform with clear KPIs for engagement, lead conversion, and assistant answer quality.

2. Scope, Technology Stack & Architecture

The platform is a decoupled (headless) web application: a component-based front end, a headless CMS for marketing content, a lightweight API/back-end for forms and bookings, and an isolated AI-assistant service. This separation lets the website, back end, and conversational AI evolve and be tested independently.

2.1 Recommended technology stack

Layer	Technology (recommended)	Purpose
Front end	Next.js (React) + TypeScript, Tailwind CSS	SSR/SSG marketing pages, SEO, fast first paint, component reuse
CMS	Headless CMS (Strapi / Sanity / WordPress headless)	Editors manage services, blog, testimonials without code
Back end / API	Node.js (NestJS) or Python (FastAPI)	Contact/lead forms, booking, auth, assistant orchestration
Database	PostgreSQL + Redis (cache/session)	Leads, bookings, conversation logs, rate limiting
AI assistant	LLM API (e.g. Claude/GPT) + RAG over a vector DB (pgvector / Pinecone)	Grounded Q&A, service guidance, lead qualification
Infra / DevOps	Docker, GitHub Actions CI/CD, cloud host (Vercel/AWS)	Reproducible builds, automated deploys, environments
Quality	Jest, Playwright/Cypress, axe-core, k6, Promptfoo/Ragas	Unit, E2E, accessibility, load, and chatbot eval
Analytics	GA4 / privacy-first analytics + custom events	Funnels, engagement, assistant KPIs

2.2 High-level architecture

The browser loads the Next.js front end, which renders CMS-managed content and mounts the chat widget. User form submissions and bookings hit the API layer, which persists to PostgreSQL and notifies staff. Chat messages are routed to the AI-assistant service, which retrieves relevant CUS content from the vector store, constructs a grounded prompt, calls the LLM, and can trigger tool actions (capture lead, propose booking slot, hand off to a human). Every conversation is logged for evaluation and continuous improvement.

Request path	Components involved
Page view	Browser → CDN → Next.js (SSG/SSR) → Headless CMS
Lead / booking	Front end → API (FastAPI/NestJS) → PostgreSQL → email/CRM notification
Chat message	Widget → Assistant service → Vector search (RAG) → LLM → tool actions → response + log

3. Team & Responsibilities

A small cross-functional team is assumed. Roles may be combined for a lean build; the RACI below reflects primary ownership per discipline.

Role	Primary responsibilities
Product / Project Lead	Scope, backlog, stakeholder sign-off, phase gate approvals
UX/UI Designer	Research, wireframes, design system, prototypes, handoff
Frontend Engineer(s)	Next.js components, responsive build, accessibility, widget integration
Backend Engineer	APIs, database, forms, bookings, auth, integrations
ML / AI Engineer	RAG pipeline, prompt design, tools, guardrails, evaluation harness
QA Engineer	Test plans, automation, accessibility/perf/security testing, chatbot eval
DevOps (part-time)	CI/CD, environments, monitoring, secrets, deployment

4. Six-Month Timeline at a Glance

Tracks overlap deliberately. Design leads early, engineering ramps from Month 2, the AI assistant is built Months 3–4, and testing intensifies Months 5–6. The matrix marks the dominant activity per track per month (● primary, ■ supporting).

Track	M1	M2	M3	M4	M5	M6
Discovery & UX/UI Design	●	●	■			
Frontend & Backend Coding		●	●	●	■	■
AI Assistant / Chatbot Build		■	●	●	■	
Website QA & Testing		■	■	■	●	●
Chatbot Evaluation & Tuning			■	■	●	●
Launch & Post-launch					■	●

● primary focus ■ supporting / preparatory activity

4.1 Key milestones & exit gates

Month	Milestone	Exit gate (must pass to proceed)
M1	Approved designs & design system	Stakeholder sign-off on prototype; tech stack finalized
M2	Front-end shell + CMS live in staging	Core pages render from CMS; CI pipeline green
M3	All site features complete; RAG prototype	Forms/bookings work end-to-end; assistant answers seed FAQs
M4	AI assistant feature-complete & integrated	Widget live in staging with tools + guardrails
M5	Full QA + chatbot evaluation complete	Test suites pass; eval scores meet thresholds; UAT signed
M6	Production launch + monitoring	Go-live checklist complete; rollback ready; KPIs tracked

5. Phase-by-Phase Plan

Each phase below lists objectives, detailed activities, engineering deliverables, and the exit criteria that gate progression to the next month.

Month 1 — Discovery & UX/UI Design

Design track leads. Establish requirements, brand, information architecture and validated designs.

Objectives

- Confirm goals, audiences (job seekers, hiring managers, businesses) and success metrics.
- Define the information architecture mirroring the current site and plan where the assistant appears.
- Produce a reusable design system aligned to the CUS brand (navy/blue/pink, globe logo).

Key activities

- Stakeholder workshops; audit of the existing cussolution.com content, analytics and gaps.
- Competitive and accessibility review; define WCAG 2.1 AA targets and performance budgets.
- Low-fidelity wireframes for all page types → high-fidelity mockups in Figma; interactive prototype.
- Design the chat-assistant UX: entry point, welcome state, suggested prompts, hand-off and error states.
- Content plan and CMS content model (services, blog, testimonials, team, FAQs for the assistant).

Deliverables

- Approved sitemap and user flows; Figma design system (typography, color tokens, components).
- High-fidelity mockups for Home, About, four Service pages, Our Approach, Blog, Contact.
- Chat-widget UX spec and copy guidelines; content model documentation.
- Finalized technical stack decision and repository/environment scaffolding.

Exit criteria

- Stakeholder sign-off on the prototype and design system.
- Backlog groomed into epics/stories with estimates for Months 2–4.

Month 2 — Front-End Foundation & Back-End Setup

Engineering ramps. Convert designs into a working, CMS-driven site shell with CI/CD.

Objectives

- Stand up the Next.js + TypeScript project, design-system components, and headless CMS.
- Establish the back-end skeleton, database schema, and automated pipeline.
- Reach a deployable staging environment rendering real content.

Key activities

- Implement the component library (buttons, cards, nav, service blocks, testimonial slider, footer) from the design tokens.
- Build responsive layouts for all page templates; wire pages to the headless CMS.
- Set up PostgreSQL schema (leads, bookings, content refs, conversation logs) and the API scaffold.
- Configure GitHub Actions: lint, type-check, unit tests, build, and deploy-to-staging on merge.
- Add SEO essentials (metadata, sitemap, structured data), analytics scaffolding, and error monitoring.

Deliverables

- Component library in Storybook; all page templates rendering CMS content in staging.
- CI/CD pipeline with automated staging deploys; base unit-test coverage in place.
- Database schema v1 and API scaffold with health checks.

Exit criteria

- All core pages viewable in staging and populated by editors via the CMS.
- CI pipeline green; Lighthouse baseline captured.

Month 3 — Core Feature Development & RAG Prototype

Peak coding. Complete interactive site features and stand up the assistant's retrieval prototype.

Objectives

- Finish all dynamic website functionality (forms, bookings, blog, search).
- Build the AI-assistant knowledge base and a working RAG question-answer prototype.

Key activities

- Implement contact/lead forms with validation, spam protection, and CRM/email notification.
- Build the consultation booking flow (availability, confirmation, calendar/email integration).
- Complete blog listing/detail, category filtering, testimonials, and site search.
- Ingest CUS content (services, FAQs, blog, policies) → chunk, embed, and load into the vector store.
- Implement the retrieval pipeline and a first grounded prompt; expose a developer test endpoint.
- Write unit and integration tests alongside features (test-as-you-build).

Deliverables

- Fully functional website in staging: forms, bookings, blog, and search all working end-to-end.
- Vector knowledge base + retrieval service answering a seed FAQ set with citations.
- Growing automated test suite (unit + integration) for site features.

Exit criteria

- End-to-end lead and booking flows verified in staging.
- RAG prototype answers seed questions accurately enough to proceed to full assistant build.

Month 4 — AI Assistant / Chatbot Build & Integration

Turn the RAG prototype into a production-grade, tool-using assistant embedded in the site.

Objectives

- Deliver a feature-complete conversational assistant with guardrails and actions.
- Integrate the chat widget into the live site experience across devices.

Key activities

- Build the assistant service: conversation state, system prompt, grounding, and streaming responses.
- Add tool/function calling: capture a lead, suggest a service, propose a booking slot, and human hand-off.
- Implement guardrails: scope limiting to CUS topics, refusal/safety rules, PII handling, and rate limiting.
- Integrate the front-end chat widget (from the Month 1 UX spec) with suggested prompts and fallbacks.

- Log every conversation with metadata (retrieved sources, latency, tokens) for evaluation.
- Add graceful degradation: fallback message and contact-form hand-off if the model is unavailable.

Deliverables

- Assistant live in staging: grounded answers, tool actions, guardrails, and conversation logging.
- Chat widget integrated and responsive across breakpoints.
- Admin view (or export) of conversation logs for review.

Exit criteria

- Assistant handles the defined intents (service Q&A, lead capture, booking, hand-off) in staging.
- Guardrails demonstrably block out-of-scope and unsafe requests.

Month 5 — Full QA, Chatbot Evaluation & UAT

Testing intensifies across both the website and the assistant; user acceptance testing runs in parallel.

Objectives

- Validate the entire platform against functional, non-functional, and accessibility requirements.
- Run a rigorous, metrics-driven evaluation of the chatbot and tune it to meet quality thresholds.

Key activities

- Execute the full website test matrix: E2E user journeys (Playwright/Cypress), cross-browser/device, regression.
- Non-functional testing: performance/load (k6), Lighthouse budgets, security (OWASP checks, dependency scan), SEO.
- Accessibility audit (axe-core + manual screen-reader/keyboard passes) to WCAG 2.1 AA.
- Chatbot evaluation: build a gold-set of representative questions; score groundedness/faithfulness, relevance, and answer correctness (Ragas/Promptfoo).
- Adversarial and safety testing of the assistant: prompt injection, jailbreaks, out-of-scope, hallucination probes, PII leakage.
- Tune prompts, retrieval (chunking, top-k, reranking) and guardrails; re-run evals to confirm gains; conduct UAT with stakeholders.

Deliverables

- Test reports for functional, performance, security, and accessibility suites with defects triaged and fixed.
- Chatbot evaluation report with scores vs. thresholds, adversarial results, and remediation.
- Signed UAT acceptance and a prioritized go/no-go defect list.

Exit criteria

- All critical/high defects resolved; automated suites green in CI.
- Chatbot meets target thresholds (e.g. groundedness and answer-relevance above agreed bar; zero critical safety failures).
- UAT sign-off obtained.

Month 6 — Launch, Hardening & Post-Launch Optimization

Controlled production launch, monitoring, and a measured optimization loop.

Objectives

- Deploy the website and assistant to production safely, with monitoring and rollback.
- Establish the continuous-improvement loop for both the site and the AI assistant.

Key activities

- Complete the go-live checklist: DNS/SSL, CDN, backups, secrets, environment parity, and legal/privacy pages.
- Production deployment via CI/CD with a rollback plan; optional canary/soft launch to a subset of traffic.
- Set up monitoring/alerting: uptime, error rates, latency, Core Web Vitals, and assistant health (cost, latency, fallback rate).
- Enable analytics dashboards and assistant KPI tracking; document runbooks and support handover.
- Post-launch: review real conversation logs weekly, expand the knowledge base, and refine prompts/guardrails.
- Backlog of enhancements (personalization, multilingual support, CRM deepening) for the next cycle.

Deliverables

- Live production website and AI assistant with monitoring and documented runbooks.
- Analytics + KPI dashboards; conversation-review and content-update process in operation.
- Post-launch report and prioritized roadmap for the following quarter.

Exit criteria

- Stable production operation against SLOs; KPIs being tracked and reported.
- Documented handover complete; improvement loop running.

6. AI Assistant — Technical Design

The assistant is a Retrieval-Augmented Generation (RAG) system so that answers stay grounded in CUS Solution's real services and content instead of the model's general knowledge. This reduces hallucination, keeps information current (update content, not the model), and lets answers cite their source.

6.1 Pipeline

- **Ingestion:** CUS pages, service descriptions, FAQs, blog, and policies are cleaned, chunked, embedded, and stored in a vector database (pgvector or Pinecone) with metadata.
- **Retrieval:** a user question is embedded; the top-k most similar chunks are retrieved (optionally reranked) as context.
- **Generation:** a system prompt + retrieved context + conversation history is sent to the LLM, which produces a grounded, on-brand answer.
- **Tools/actions:** function calling lets the assistant capture a lead, recommend a service, offer a booking slot, or hand off to a human.
- **Guardrails:** scope limiting, safety/refusal rules, PII redaction, and rate limiting wrap the model.
- **Logging & feedback:** every turn (with sources, latency, cost, thumbs-up/down) is logged to drive evaluation and improvement.

6.2 Guardrails & safety

- Constrain the assistant to CUS-relevant topics (careers, services, staffing, booking); politely decline unrelated requests.
- Never fabricate specifics (pricing, guarantees, legal/immigration advice); defer to a human where appropriate.
- Detect and avoid storing sensitive PII; comply with privacy policy and consent.
- Resist prompt injection and jailbreak attempts; fail closed to a safe fallback and contact hand-off.

7. Testing Strategy

Two coordinated programs run: conventional software QA for the website, and a specialized evaluation program for the conversational AI. Testing is continuous (in CI from Month 2) and culminates in the dedicated Month 5 phase.

7.1 Website QA matrix

Test type	Tooling	What it verifies
Unit	Jest / Vitest	Components and functions behave in isolation
Integration	Testing Library / Supertest	Modules and API endpoints work together
End-to-end	Playwright / Cypress	Real user journeys: browse, submit lead, book
Accessibility	axe-core + manual	WCAG 2.1 AA: keyboard, screen reader, contrast
Performance	Lighthouse CI / k6	Core Web Vitals and load behavior vs. budgets
Security	OWASP ZAP, dependency scan	Common vulns, headers, secrets, input handling
Cross-browser	BrowserStack / device lab	Consistent rendering on browsers and devices
Regression	CI suite	New changes don't break existing features

7.2 Chatbot evaluation program

The assistant is tested against a curated gold-set of representative candidate and client questions, plus adversarial cases. Quality is measured, not eyeballed, and thresholds gate release.

Dimension	Method / metric
Groundedness / faithfulness	Answer is supported by retrieved CUS content (Ragas faithfulness)
Answer relevance	Response addresses the question (Ragas answer-relevance)
Retrieval quality	Context precision/recall — did we fetch the right chunks?
Correctness	Compared to expected answers on the gold-set (human + LLM-as-judge)
Safety / adversarial	Prompt injection, jailbreak, out-of-scope, PII leakage, hallucination probes
Tone & brand	On-brand, helpful, concise; rubric-scored samples
Operational	Latency, cost per conversation, fallback rate, containment/hand-off rate

Evaluations are automated (Promptfoo / Ragas) and re-run on every prompt, retrieval, or model change so regressions are caught before release — the conversational analogue of a CI test suite.

8. Risks & Mitigations

Risk	Impact	Mitigation
Assistant hallucination / wrong info	High	RAG grounding, citations, guardrails, eval thresholds, human hand-off
Scope creep in design/features	Med	Phase gates, groomed backlog, change control
Content not ready for CMS/KB	Med	Content plan in M1; parallel content workstream
LLM cost/latency in production	Med	Caching, retrieval limits, model selection, monitoring/alerts
Privacy / PII exposure	High	PII redaction, consent, data-retention policy, security testing
Timeline slippage	Med	Overlapping tracks, weekly checkpoints, buffer in M6

9. Success Metrics (KPIs)

- **Engagement:** bounce rate, pages/session, average session duration, and Core Web Vitals (LCP, CLS, INP).
- **Conversion:** contact-form submissions, consultation bookings, and lead-to-booking rate.

- **Assistant usage:** chat sessions, messages/session, and % of visitors engaging the assistant.
- **Assistant quality:** groundedness/answer-relevance eval scores, thumbs-up rate, containment (resolved without human), and hand-off rate.
- **Operational:** uptime/SLO adherence, p95 latency, and cost per conversation.

10. One-Page Summary

Month	Theme	Headline deliverable
1	Design	Approved designs, design system, IA, assistant UX spec
2	Coding (foundation)	CMS-driven site shell in staging + CI/CD
3	Coding (features) + AI	Full site features + RAG prototype
4	AI assistant	Feature-complete, integrated chatbot with guardrails
5	Testing	Full QA + chatbot evaluation + UAT sign-off
6	Launch	Production launch, monitoring, optimization loop

Prepared for CUS Solution. This plan assumes overlapping design, engineering and QA tracks with phase-gate approvals. Durations are indicative and should be confirmed against final team size and scope.